



Shiftbase QA Assessment

End-to-end automation of the signup & login flows

Two flows, automated to the dashboard — plus eleven real defects the automation surfaced along the way.

Playwright + TypeScript, on the element-interactions Steps API. Runs green from a cold clone with no configuration.

The brief, and what shipped

Automate two flows against the Shiftbase demo environment: signup + onboarding up to the dashboard, and login. This is what came back.



Both flows, end to end

Signup walks the 8-step questionnaire and setup wizard to the dashboard; login handles the MFA interstitial. Interactions read as intent through the Steps API — no raw selectors in any test.



Eleven defects found

Not the point of the brief, but the honest by-product of testing a real product: three found while building, eight from a dedicated adversarial pass. Zero were false positives.



Evidence, not claims

Every defect carries a reproduction and screen recording / screenshot. Findings that could not be pinned by a portable test are reported with the reason, not faked into one.



Runnable by anyone

`npm ci && npm test` on a clean clone with no `.env`. Verified by cloning fresh; the suite mints its own account through the real signup flow.

Coverage at a glance

29 tests across three Playwright projects (desktop, mobile, and an account-provisioning setup project).

23

passing tests

6

failing by design

6

pages mapped

56

elements

11

defects found



The 6 failing tests fail on purpose

Each is tagged `@known-defect` and describes the behaviour the product *should* have — so it fails today and becomes a regression guard the day the bug is fixed. `npm run test:no-defects` is 23 / 23 green.



Not weakened to pass

Rewriting a test to agree with a bug makes the suite unable to detect it. The failing tests stayed honest instead.

What each test protects

Chosen, not sprayed — every scenario covers a distinct way a flow can break.

FLOW	COVERAGE
Signup	Registration → questionnaire → wizard → dashboard · form handoff · mobile viewport · empty-submit & per-field validation (email, phone, weak password, terms)
Login	Valid credentials · the MFA-promote interstitial · wrong password · unknown email · identical rejection message (no account enumeration) · protected-route redirect · "Back to login" is a logout · concurrent & stale sessions
Session	Survives a reload (asserts the dashboard re-renders, not just the URL) · primary navigation present

Deliberately out of scope, and said so in the README: everything past the dashboard, password reset, cross-browser, visual regression, load.

The defects

Eleven confirmed, zero refuted. The headline is one UI testing alone would never reach.

HIGH · compliance

Terms & Data-Processing-Agreement acceptance is enforced only in the browser

POST /api/signup accepts a payload with `accept_conditions: false` and creates a fully usable account — the required consent checkbox has no server-side counterpart. Reproduced from two fresh sessions, each followed by a successful login. Reported, not automated: a test would replay a compliance-bypass exploit on every CI run against a shared environment.

SOURCE	FINDINGS	AUTOMATED
Found while building	Router blank-page on a bad modal route · duplicate-email accepted then password rejected · unbounded name length	2 of 3 as tests
Adversarial pass	Terms bypass · no login rate-limit · timing enumeration · gateway error shown as bad credentials · swallowed reset message · fractional employee count · missing Back button · raw-500 on malformed API	2 as tests, 6 reported

How the findings were trusted

Every adversarial finding was reproduced, then handed to a second, independent check whose only job was to disprove it.



- ✓ **Selectors that survive copy changes**
The questionnaire's radio inputs are unclickable (`sr-only`); options are targeted by stable `formcontrolname + value`, not the visible label — which disagrees with the value ("Scheduler" is `Planner`).
- ✓ **App bugs reported, never worked around**
When the app misbehaved, the test was left describing correct behaviour and the bug was written up — not adjusted to match.

Engineering choices worth a look

The decisions a reviewer running this cold would notice.



No committed credentials

A setup project mints an account via the real signup flow when no `.env` is present, so a cold clone runs green. Secrets live in `.env` (gitignored); an independent sweep confirmed none leaked.



Flake, root-caused not retried

The Angular questionnaire drops clicks mid-render; every input is now re-applied until it lands (verified 12× with retries off). The residual is the app's own gateway flakiness, left to configured retries.



Watchable headed runs

`npm run test:headed` drops to one worker and paces each action, so a reviewer can actually follow the flows instead of windows racing.



TLS reality documented

The demo serves a private-CA cert that rolls every 24h; the suite sets `ignoreHTTPSErrors` and says so plainly — on production that setting would hide a real defect.

Ready to run, honest about its edges

The README carries the full reasoning, the findings, and a candid “what I’d do next.”



What I'd do next

Raise the Terms/DPA server-side check first — small fix, real legal weight. Move account minting to the API to cut runtime. Add data cleanup for the accounts each run leaves behind.

@playwright/test

1.62

@civitas-cerebrum/element-interactions

0.3.9

TypeScript

strict